4. Python: Conditional Statements

"You do not just code. You also set conditions"

Previous Section:

</> 3. Python: Data Types

Contents:

Programs are not always meant to run the same instructions every single time. Sometimes, a program must make decisions depending on the situation. This is where Conditional Statements come in. In Python, conditional statements allow the program to check whether a condition is `True` or `False`, and based on that result, execute different parts of the code.

Think about it like making choices in real life:

> ***"If it rains, bring an umbrella. Otherwise, go outside normally."***

Programming follows the same idea. By using conditions, Python programs become dynamic, interactive, and much smarter than simple step-by-step scripts.

1. If-Else Statements

Now let's begin with the foundation of decision-making in Python: the `if-else` statement.

An `if` statement allows the program to check a condition. If the condition is true, the block of code inside the `if` statement executes. Otherwise, Python moves to the `else` block instead.

Before using conditions, we need comparison operators. These operators compare values and return either `True` or `False`.

Some of the most common comparison operators are:

- `>` Greater than
- `<` Less than

- `>=` Greater than or equal to
- `<=` Less than or equal to
- `==` Equal to *(important: this is not the same as `=`)*
- `!=` Not equal to

For example:

```
i = int(input("Enter a number: "))

if i >= 10:
    print(i, "is equal to or greater than 10")
else:
    print(i, "is not greater than 10")
```

```
Enter a number: 12
12 is equal to or greater than 10
```

Let's understand what happens here. The program first asks the user to enter a number. Since `input()` always returns a string, we convert it into an integer using `int()`.

Then Python checks the condition: `i >= 10` . If the condition is `True`, the first message is printed. Otherwise, the program executes the `else` block.

This simple structure becomes the backbone of decision-making in programming. Here are some other useful examples.

Determine Odd and Even Number

```
number = int(input("Enter a number: "))

if number % 2 != 0:
    print(number, "is an odd number")
else:
    print(number, "is an even number")
```

```
Enter a number: 7
7 is an odd number
```

Notice the use of `%` (modulus operator). The modulus operator returns the remainder after division. Even numbers leave a remainder of `0` when divided by `2`. Odd numbers do not

So: `number % 2 != 0` means: "The remainder is not zero" → therefore the number is odd.

Password Checking Example

```
password = input("Enter the password: ")

if password == "iris123":
    print("Access Granted")
else:
    print("Incorrect Password")
```

Enter the password: iris123
Access Granted

This example demonstrates the `==` operator, which checks equality.

Be very careful here: `password = "iris123"` uses `=` for assignment. But: `password == "iris123"` checks whether the two values are equal. This is one of the most common beginner mistakes in Python.

So in the end, `if-else` statements allow programs to make decisions instead of blindly executing the same instructions every time. Without conditions, programs would not be interactive, responsive, or intelligent.

2. Boolean Conditional Statements

Now let's make conditions more powerful. Sometimes, a single condition is not enough. Real-world decisions usually involve multiple requirements at once. For example:

- A student must pass both math **and** science
- A website may allow login if the username **or** email is correct
- A condition may need to be reversed using `not`

This is where Boolean operators come in.

Python provides three major Boolean operators: `and` , `or` , `not`. These operators combine or modify conditions.

The `and` Operator

The `and` operator requires both conditions to be `True`. Example:

```
x = int(input("Enter a number x: "))

if (x > 10) and (x % 2 == 0):
    print('The number is greater than 10 and is an even num
ber')
else:
    print('The number is not greater than 10 or is not an e
ven number')
```

```
Enter a number x: 14
The number is greater than 10 and is an even number
```

Let's analyze the condition: `(x > 10) and (x % 2 == 0)` → The number must satisfy BOTH conditions: "Greater than `10` " and "Even number". If one condition fails, the entire statement becomes `False`.

Think of `and` like two locked doors: Both must open for you to pass.

The `or` Operator

The `or` operator is less strict. It only requires one condition to be `True`. Example:

```
y = int(input("Enter a number y1: "))

if (y > 10) or (y % 2 != 0):
    print('The number is greater than 10 or is not an even
number')
else:
    print('The number is not greater than 10 and is an even
number')
```

Enter a number y1: 15
The number is greater than 10 or is not an even number

Here `(y > 10) or (y % 2 != 0)` means: "The number is greater than `10`" OR "The number is odd". Only one needs to be true.

Think of `or` like having two possible keys: Either key can unlock the door.

Leap Year Example

Now let's combine multiple Boolean operators into something more realistic.

```
year = int(input("Enter a year: "))

if ((year % 4 == 0) and (year % 100 != 0)) or (year % 400 =
= 0):
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

Enter a year: 2024
2024 is a leap year:

This condition may look complicated at first, but it follows the official leap year rules:

- A year is a leap year if: It "is divisible by `4` AND Not divisible by `100` " OR "It is divisible by `400` ".

This example demonstrates how Boolean operators allow Python to represent real-world logic very precisely.

The `not` Operator

The `not` operator reverses a Boolean condition. If something is `True`, `not` turns it into `False`. If something is `False`, `not` turns it into `True`. Example:

```
number = int(input("Enter a number: "))

if not (number % 2 == 0):
    print(number, "is an odd number")
else:
    print(number, "is not an odd number")
```

```
Enter a number: 19
19 is an odd number
```

Here: `number % 2 == 0` checks whether the number is even. Then: `not (number % 2 == 0)` reverses the result. So the program becomes: ***"If the number is NOT even...",*** which means the number must be odd.

Boolean operators make conditions significantly more flexible and expressive. Instead of checking only one situation at a time, we can combine multiple logical rules together. This is what allows programs to simulate real decision-making processes rather than simple one-condition checks.

3. Membership Conditional Statements

So far, our conditions mainly focused on comparisons like: Greater than; Less than; Equal to. But sometimes, we are not comparing numbers directly.

Instead, we want to ask a different kind of question: ***"Does this value exist inside a collection?"***

This is where Membership Operators become useful. Python provides two membership operators: `in` and `not in`. These operators are used to check whether a value exists inside a sequence such as: Lists; Tuples; Strings; Sets.

Instead of comparing values mathematically, membership operators search through collections of data.

The `in` Operator

The `in` operator returns `True` if a value exists inside the sequence. Think of it like asking: ***"Can I find this item somewhere inside this collection?"***

Example:

```
list_of_fruits = ['apple', 'banana', 'cherry']
fruit = 'apple'

if fruit in list_of_fruits:
    print("Yes,", fruit, "is in the list")
else:
    print("No,", fruit, "is not in the list")
```

Yes, apple is in the list

Let's understand what happened here. We created a list called: `list_of_fruits` which contains multiple values. Then Python checks: `fruit in list_of_fruits`. This means: "Does the value `'apple'` exist inside the list?" Since `'apple'` is indeed present, the condition becomes `True`, and the first message executes.

Membership operators are extremely useful when validating data, checking user input, filtering values, or searching collections.

The `not in` Operator

The `not in` operator does the opposite. It returns `True` if the value does NOT exist inside the sequence. Example:

```
month = int(input("Enter a month: "))

list_of_months = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

if month not in list_of_months:
    print(month, 'is not a month')
else:
    print('Month is', month)
```

Enter a month: 15
15 is not a month

Here, Python checks: `month not in list_of_months` . This asks: “Is the entered number missing from the list of valid months?” Since `15` does not exist in the list, the condition becomes `True`.

This type of validation is very common in programming because it helps prevent invalid inputs from entering the system.

Membership Operators with Strings

Membership operators also work with strings.

But here’s the interesting part: Python treats a string like a sequence of characters. That means we can search for words, letters, small phrases, etc. inside a larger string.

Example:

```
sentence = "This is Python Programming. Hello!! My name is Iris"

if "Python" in sentence:
    print("Python was found in the sentence")
else:
    print("Python was not found")
```

Python was found in the sentence

Python searches through the sentence and checks whether the word `"Python"` exists inside it.

We can also use `not in`. Example:

```
sentence = "This is Python Programming. Hello!! My name is Iris"

if "Java" not in sentence:
    print("Java was not found in the sentence")
else:
    print("Java exists in the sentence")
```

Java was not found in the sentence

This demonstrates something very powerful about Python: Membership operators are not limited to numbers. They also work with text, collections, and many other data structures.

So in the end, membership operators allow programs to search, validate, and verify data efficiently. Instead of manually checking every value one by one, Python can quickly determine whether something exists inside a collection using simple and readable syntax like `in` and `not in`.

4. Multiple If-Else Statements

So far, we have mainly worked with single conditions.

```
if condition:
    ...
else:
    ...
```

But real programs are rarely that simple. Sometimes, a program must evaluate multiple possible conditions at once. And depending on how we structure those conditions, Python behaves very differently.

This is where understanding Multiple If-Else Statements becomes extremely important. When Python encounters multiple conditions, it reads them from top to bottom, one by one. However, the flow changes depending on whether we use: Multiple independent `if` statements OR an `if-elif-else` chain.

These two structures may look similar, but they behave completely differently.

Non-Nested Conditions (Multiple Independent `if` Statements)

Non-Nested conditions use multiple separate `if` statements. Each condition is checked independently.

This means more than one condition can be `True` , and multiple blocks can execute at the same time, Think of it like multiple questions being asked separately.

Example:

```
# Non-Nested: Check if a number is divisible by 2, 3, or 5

number = int(input("Enter a number: "))

if number % 2 == 0:
    print(number, "is divisible by 2")

if number % 3 == 0:
    print(number, "is divisible by 3")

if number % 5 == 0:
    print(number, "is divisible by 5")

# The else only applies to the last if
else:
    print(number, "is not divisible by 2, 3, or 5")
```

```
Enter a number: 15
15 is divisible by 3
15 is divisible by 5
```

Let’s carefully analyze the flow.

The number `15` is checked against all three conditions:

- Divisible by `2` → False
- Divisible by `3` → True
- Divisible by `5` → True

Because these are separate `if` statements, Python executes BOTH matching conditions.

This is extremely important: Python does NOT stop after the first true condition when using independent `if` statements. Every condition is evaluated separately.

Important Note About `else`
Many beginners misunderstand this part:

```
else:
    print(number, "is not divisible by 2, 3, or 5")
```

This `else` only belongs to:

```
if number % 5 == 0:
```

NOT all three conditions. So if the number is not divisible by `5`, the `else` executes—even if the number was divisible by `2` or `3`.
For example:

```
Enter a number: 6
6 is divisible by 2
6 is divisible by 3
6 is not divisible by 2, 3, or 5
```

That output is logically incorrect. This is one reason why independent `if` statements must be used carefully.

Nested Conditions (`if-elif-else`)

Now let's look at the other structure. When using `if-elif-else`, Python behaves differently. Instead of checking every condition independently, Python stops immediately after finding the first `True` condition. Only ONE block executes.

Think of it like a decision hierarchy: ***"Check this first. If false, check the next. If false again, continue downward".***

Example:

```
# Nested: Determine the grade level

grade = float(input("Enter your grade: "))

if grade >= 9.5:
    print('Level A+')

elif grade >= 8.0:
    print('Level A')

elif grade >= 7.0:
    print('Level B')

elif grade >= 6.5:
    print('Level C')

elif grade >= 5.0:
    print('Level D')

# The else executes when all above conditions are false
else:
    print('Level F')
```

```
Enter your grade: 8.5
Level A
```

Let's examine the flow carefully. Python starts from the top:

- `8.5 >= 9.5` → False

- `8.5 >= 8.0` → True

The moment Python finds a true condition, it executes that block and immediately stops checking the remaining conditions.

Even though `8.5 >= 7.0` is also true, Python ignores it completely because an earlier condition already matched.

This behavior makes `if-elif-else` perfect for situations where only ONE final result should occur.

The Difference Between Non-Nested and Nested Conditions

This distinction is one of the most important concepts in conditional statements. At first glance, these structures may appear similar, but their execution flow is completely different.

- **Example Using Multiple Independent `if`**

```python
age = int(input("Enter your age: "))

if age > 6:
    print("youngster")

if age > 12:
    print("teenager")

if age > 18:
    print("adolescent")

if age > 30:
    print("adult")

if age > 60:
    print("elder")

else:
    print("unknown")
```

```
Enter your age: 65
youngster
teenager
adolescent
adult
elder
```

Why we end up with this output? Because every condition is checked independently. A person aged `65` satisfies ALL those conditions.

And again, `else:` only belongs to the last condition `if age > 60:`

- **Example Using `elif`**

Now compare it with this version:

```python
age = int(input("Enter your age: "))

if age > 60:
    print("elder")

elif age > 30:
    print("adult")

elif age > 18:
    print("adolescent")

elif age > 12:
    print("teenager")

elif age > 6:
    print("youngster")

else:
    print("unknown")
```

```
Enter your age: 65
elder
```

This time round, Python stops immediately after the first true condition. Only one result appears.

- **Understanding the Flow - This is the real difference:**

Independent `if`

```python
if ...
if ...
if ...
```

Every condition is checked → Multiple results may execute → `else` only belongs to the nearest `if`

`if-elif-else`

```
if ...
elif ...
elif ...
else ...
```

Conditions are checked in order → Only the FIRST true condition executes → Remaining conditions are ignored → `else` ***handles all unmatched cases***

So in the end, choosing between Non-Nested and Nested conditions depends entirely on the logic of the program. Use multiple independent `if` statements when multiple conditions can all be true simultaneously. Use `if-elif-else` when only one final outcome should happen. Understanding this flow is essential because many logical programming errors come not from syntax mistakes, but from misunderstanding how conditions execute internally.

5. Logical Built-in Functions

Up until now, we mainly used conditional statements with operators such as: `>` , `<` , `==` , `and` , `or` . But Python also provides powerful built-in functions that simplify logical checking across entire collections of data.

Two of the most useful logical built-in functions are: `all()` and `any()`

These functions work with iterable sequences such as: Lists; Tuples; Sets; Strings. Instead of manually checking every value one by one, Python can automatically evaluate multiple conditions across the sequence.

The `all()` Function

The `all()` function checks whether ALL conditions inside the iterable are `True`. If even one condition becomes `False`, the entire result becomes `False`.

Think of it like a strict inspection: **“Everything must pass”.**

Example idea: `all(values)` returns: `True` → if every value is true; `False` → if at least one value is false

The `any()` Function

The `any()` function is less strict. It checks whether AT LEAST ONE condition is `True`.

Think of it like a minimum requirement: ***"If at least one exists, then it succeeds".***

Example idea: `any(values)` returns: `True` → if one or more values are true; `False` → if everything is false

Grade Evaluation Example

Now let's combine `all()` and `any()` into a realistic grading system.

```python
score = [8.4, 7.5, 8.4, 4.7, 4.1, 5.8, 9.1, 8.5]

average = sum(score) / len(score)

print(f"Your average is {average:.1f}")

if average >= 8.0:
    print('Your average is 8.0 or above')

    if all(s >= 6.5 for s in score):
        print('All of the subject scores are 6.5 and abov
e')
        print('Your final grade is A+')

    else:
        print('Not all of subject scores are 6.5 and abov
e')
        print('Your final grade is A')

else:
    print('Your average is lower than 8.0')

    if any(s < 5.0 for s in score):
        print('There is at least one subject below 5.0')
        print('Your final grade is C')

    else:
        print('Your final grade is B')
```

Your average is 7.1
Your average is lower than 8.0
There is at least one subject below 5.0
Your final grade is C

Understanding the Workflow

Let's break the logic down carefully.

We first create a list of scores: `score = [8.4, 7.5, 8.4, 4.7, 4.1, 5.8, 9.1, 8.5]`

Then calculate the average: `average = sum(score) / len(score)`

Here:

- `sum(score)` adds all scores together
- `len(score)` counts how many subjects exist

The average becomes: `7.1` . Since the average is lower than `8.0`, Python moves into the `else` block.

Now the program checks: `any(s < 5.0 for s in score)` . This line may look complicated at first, but the idea is actually simple.

Python checks every score one by one:

- Is this score below `5.0`?
- Is this one below `5.0`?
- Is another one below `5.0`?

The moment Python finds at least one matching value, `any()` becomes `True`.

Since: `4.7` and `4.1` are below `5.0`, the condition succeeds. Therefore, this is what the program prints:

```
There is at least one subject below 5.0
Your final grade is C
```

Understanding `all()`

Now imagine the average was above `8.0`. Python would execute: `all(s >= 6.5 for s in score)`

This checks: “Are ALL subject scores greater than or equal to `6.5`?” If even one subject is below `6.5`, the condition fails.

At the end of the day:

- `all()` requires everything to pass
- `any()` only requires one match

This distinction is extremely important in logical programming.

Without `all()` and `any()`, we would need many repetitive conditional statements and loops. These functions allow us to validate datasets, check

requirements, detect failures, and search for conditions efficiently. These functions are widely used in the field of Data Analytics and Grade Management System.

This is only the beginning. Python contains many more built-in functions that simplify conditions in programming significantly. Some of them will be introduced in future sections.

Summary

Conditional Statements are one of the most important foundations in programming—not just in Python, but in almost every programming language (even spreadsheet systems like Excel).

They allow programs to make decisions instead of blindly executing instructions from top to bottom.

In this section, we explored multiple forms of conditional logic.

- We started with the basic `if-else` structure, where programs execute different blocks depending on whether a condition is `True` or `False`. From there, we expanded into Boolean Conditional Statements using operators such as: `and` , `or` , `not` , which allow multiple logical conditions to work together.
- After that, we explored Membership Conditional Statements using: `in` and `not in` to check whether values exist inside collections such as lists or strings.
- Then we moved into Multiple If-Else Statements, where we learned the major difference between: Independent `if` statements and `if-elif-else` chains. One checks every condition independently, while the other stops after the first matching condition.
- Finally, we introduced logical built-in functions: `all()` and `any()`, which allow Python to evaluate entire collections of conditions efficiently.

Together, all these concepts form the backbone of decision-making in programming. Without conditional statements:

- Programs cannot validate inputs
- Systems cannot respond dynamically
- Applications cannot make logical choices

- Intelligence in software becomes impossible

So in many ways, Conditional Statements are what transform a simple script into an actual interactive program capable of reasoning and reacting to different situations.

References

https://www.geeksforgeeks.org/python/conditional-statements-in-python/

https://www.geeksforgeeks.org/python/any-all-in-python/

https://builtin.com/data-science/python-conditional-statement

Next Section:

</> 5. Python: Operators